

R4.01 - Architecture Logiciel

1. Moteur de production (cargo)	1
2. Types de base	1
3. Structures	2
3.1 Structures tuples	2
4. Enumération	3
4.1 Option<T> et Result<T>	3
5. Traits (Interfaces)	4
6. Fonctions anonymes	4
7. Options en ligne de commande	4

1. Moteur de production (cargo)

Créer un projet my_project	cargo new my_project --lib
Ajouter la dépendance clap avec la fonctionnalité derive	cargo add clap --features derive
Compiler le projet	cargo build
Lancer les tests	cargo test
Lancer le programme courant avec des paramètres	cargo run --my_param value

2. Types de base

type Primitifs

Type	tailles en bites	contenu
char	32	caractère unicode
bool	8	booléen
i32	32	entier signé
usize	64 (taille des pointeurs)	entier non signé (taille, indice...)
f64	64	nombre flottant à double précision

Pointeurs

```
let x = 3;           // i32
let ptr_x = &x;      // &i32
let y = *ptr_x;      // i32
```

3. Structures

Définir une structure Point contenant deux flottants abscissa et ordinate .	<pre>struct Point { abscissa: f64, ordinate: f64, }</pre>
Instancier un objet point de type Point ayant comme coordonnées (1.0, 2.0)	<pre>let point = Point { abscissa: 1.0, ordinate: 2.0, };</pre>
Définir un constructeur new() pour le type Point prenant deux flottants en paramètres.	<pre>impl Point { fn new(x :f64, y:f64) -> Self { return Self { abscissa: x, ordinate: y, }; } }</pre>
Définir un accesseur get_abscissa() pour le type Point	<pre>impl Point { fn get_abscissa(&self) -> f64 { return self.abscissa; } }</pre>

3.1 Structures tuples

Définir une structure tuple Value contenant un flottant value	<pre>struct Value (value: f64)</pre>
Créer une instance my_value de la structure tuple Value	<pre>let my_value = Value(3.14);</pre>
Récupérer la valeur contenue dans la structure tuple my_value	<pre>let x = my_value.0;</pre>

4. Enumération

```
pub enum Option<T> {  
    None,  
    Some(T),  
}
```

4.1 Option<T> et Result<T>

Filtrer my_option de type Option<T> par match	<pre>match my_option { None => ..., Some(value) => ..., }</pre>
Filtrer my_result de type Result<T> par match	<pre>match my_option { Err(error) => ..., Ok(value) => ..., }</pre>
Récupérer la valeur dans my_result de type de type Result<T> ou retourne une erreur	<pre>fn my_function() -> Result<()> { let x = my_result?; Ok(()) }</pre>

5. Traits (Interfaces)

Définir une interface AddOne ayant une méthode increment_abscissa().	<pre>trait AddOne { fn increment_abscissa(&mut self); }</pre>
Implémenter le trait AddOne pour le type Point	<pre>impl AddOne for Point { fn increment_abscissa(&mut self) { self.abscissa += 1.0; } }</pre>
Dériver automatiquement le trait Clone pour le type Point 😊	<pre>#[derive(Clone)] struct Point {...}</pre>

6. Fonctions anonymes

Fonction anonyme décidant si son argument est pair	<pre> x x % 2 == 0</pre>
--	---------------------------

7. Options en ligne de commande

Importer clap et utiliser Parser (qui vient de clap)	<pre>use clap::Parser;</pre>
Définir une structure Parameters contenant comme arguments en ligne de commande un tableau d'entiers et optionnellement une chaîne de caractères	<pre>#[derive(Parser)] struct Parameters { #[arg(short, long, required = true, num_args = 1..)] numbers: Vec<i32>, #[arg(short, long)] word: String, }</pre>

Créer un objet
my_parameters contenant
les arguments en ligne de
commande

```
let my_parameters = Parameters::parse();
```